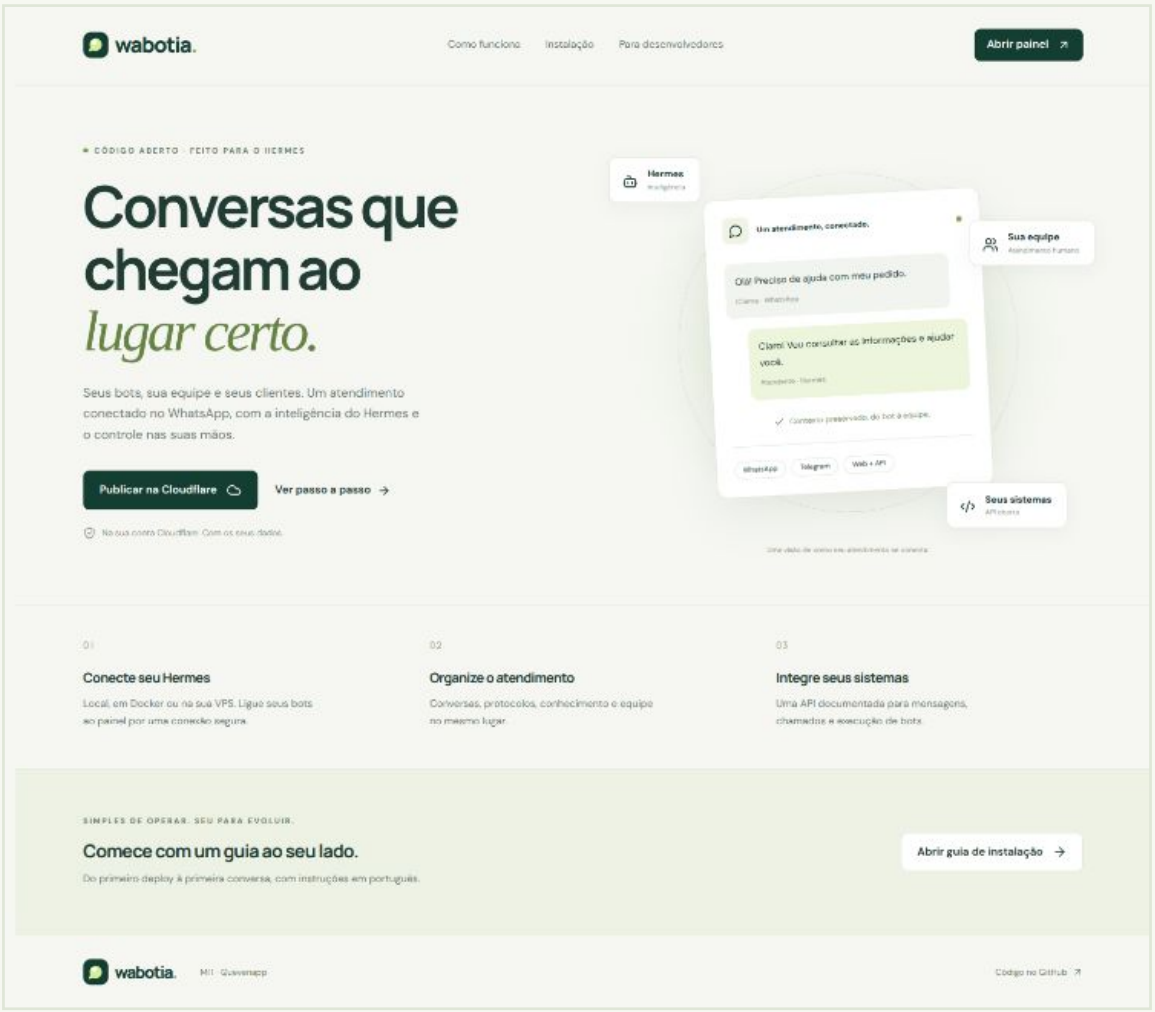
Seu atendimento. Bem conectado.

Guia para instalar na Cloudflare, conectar o Hermes e preparar sua equipe.



Página inicial do Wabotia. As ilustrações e conversas de exemplo são fictícias.

Começar em wabotia.queven.app/install

Edição de 04/11/2026. Leia na ordem; escolha apenas um modo de WhatsApp por número. O painel fica na Cloudflare. Hermes e QR continuam no computador ou servidor que você escolher.

Antes de começar

Você vai criar sua própria instalação. O site público é a porta de entrada para o guia e o código; seus atendimentos ficam nos recursos da sua conta Cloudflare.

Onde fica	O que faz
Cloudflare	Painel, API, contas, conversas, protocolos e arquivos privados. Workers + D1 + R2 + Durable Objects.
Hermes externo	Executa os bots, modelos e ferramentas já configurados. Pode estar em Linux, Windows/WSL2, macOS, Docker ou VPS.
Conector QR, se escolhido	Mantém a sessão WhatsApp em um processo externo. Precisa ficar ligado.
Cloudflare Tunnel + Access	Liga o Worker ao Hermes privado, com autenticação. Use quando a API não tem HTTPS acessível.

Tenha em mãos

1. Contas GitHub e Cloudflare. Considere Workers Paid para produção e habilite R2. Custos dependem do plano e do uso; modelos e canais são cobrados separadamente.

2. Hermes funcionando, com os perfis de bot que deseja usar. Preserve os bots e os dados existentes.

3. Um número de WhatsApp reservado ao teste e, se necessário, um bot Telegram novo e grupo privado com tópicos.

4. Um gerenciador de senhas. Guarde duas chaves aleatórias diferentes de 64 caracteres: **MASTER_KEY** e **SETUP_TOKEN**.

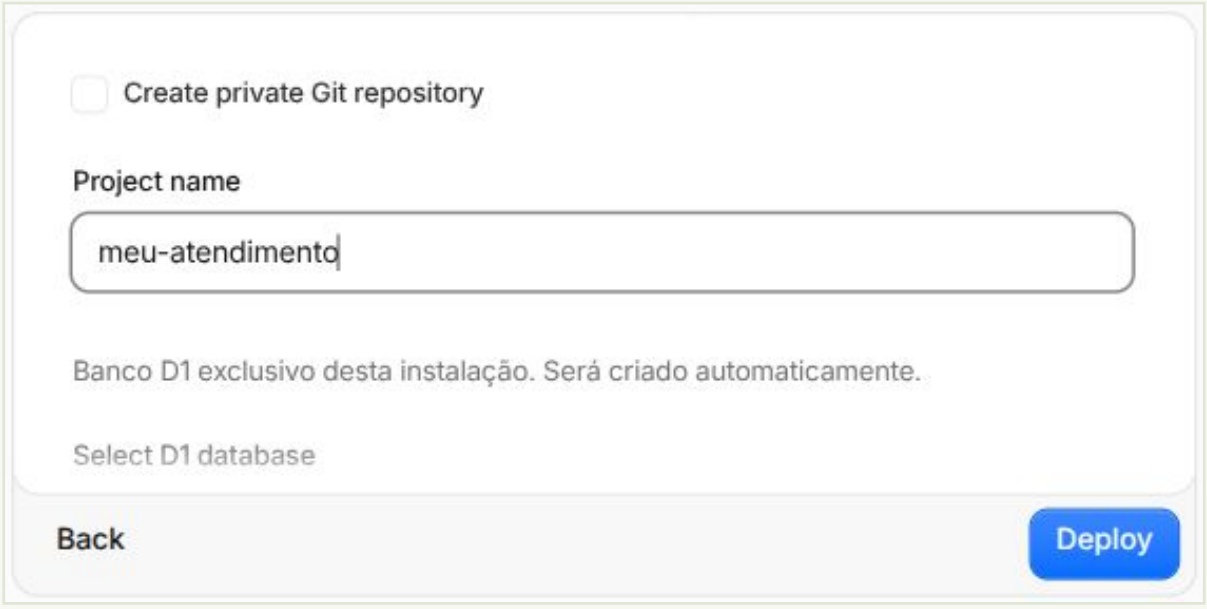
Escolha o caminho: botão na Cloudflare nas páginas 3-4, ou instalador pelo terminal na página 5. Ambos publicam o mesmo Wabotia. Não é preciso contratar VPS nem Supabase para o painel.

MASTER_KEY protege suas credenciais; perder essa chave impede restaurá-las. SETUP_TOKEN autoriza a primeira conta. A senha do administrador é escolhida separadamente.

Publicar pelo botão

1. Abra wabotia.queven.app/install e clique **Deploy to Cloudflare**. Faça login na sua conta.

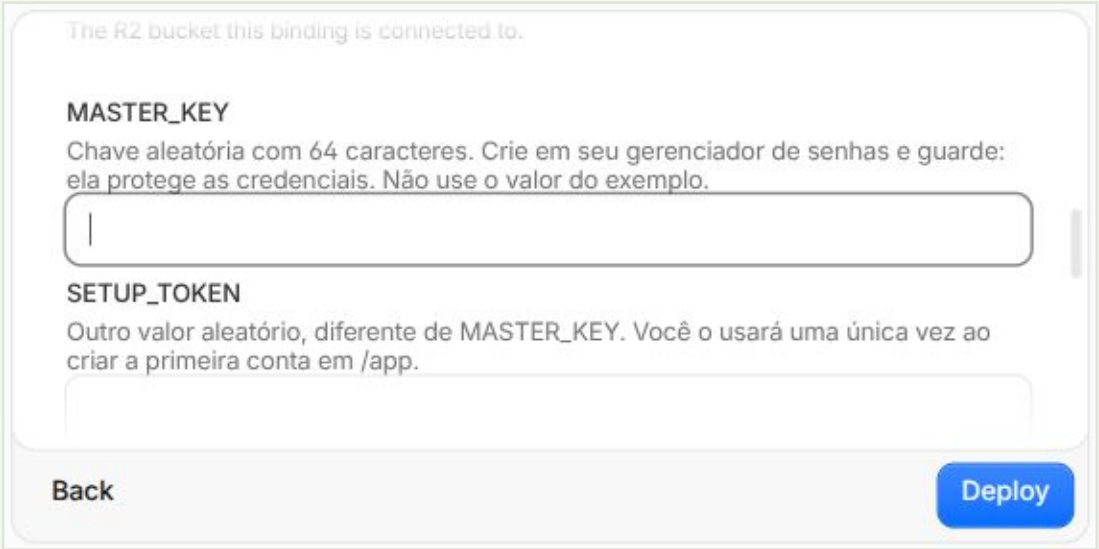
2. Confira a origem **Quevenapp/Wabotia**. Escolha sua conta GitHub em **Git account** e um nome de projeto. Será criada uma cópia do código na conta escolhida.



Tela real da Cloudflare, com nome fictício. Role dentro do formulário para ver os próximos campos.

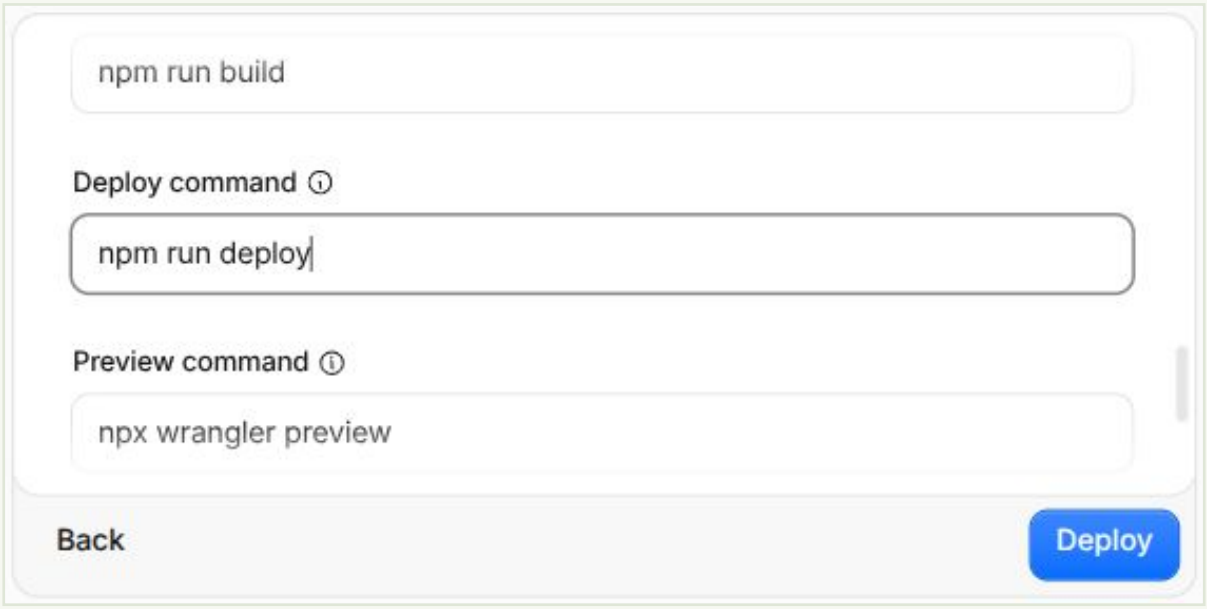
3. Use um banco D1 e um bucket R2 exclusivos desta instalação. Mantenha o bucket privado. Para uma segunda instalação, escolha recursos diferentes.

4. Preencha **MASTER_KEY** e **SETUP_TOKEN** com os valores que guardou. Não use os mesmos valores para produção e teste.



Os dois campos ficam vazios nesta referência. Nunca publique uma captura com suas chaves.

Concluir o deploy



Mantenha os comandos detectados pelo repositório. O formulário foi conferido no Chrome.

1. Em **Build command**, use **npm run build**. Em **Deploy command**, use **npm run deploy**: o comando publica o Worker e aplica as migrações D1.

2. Desative **Enable Preview builds** até preparar recursos isolados para testes. Nunca reutilize o banco de produção para experimentar mudanças.

3. Clique **Deploy** e aguarde a indicação de sucesso. Guarde a URL que termina em **workers.dev**.

4. No Worker, abra **Settings > Variables and Secrets**. Confira a existência de MASTER_KEY e SETUP_TOKEN. Se não foram solicitados, adicione ambos como **Secret**.

5. Abra a URL da **sua instalação**, acrescente **/app** e siga para a página 6.

GitHub Actions: o repositório original mantém Actions desativado. Na sua cópia, confira Settings > Actions > General > Disable actions. O botão usa Cloudflare Builds, que é outro serviço.

O fluxo do botão reconheceu recursos, secrets e comandos. A publicação completa validada nesta versão foi feita por Wrangler. Se houver erro de R2, habilite o serviço na conta; se o build falhar, confira os comandos antes de tentar outra instalação.

Alternativa: instalador

Este caminho gera as chaves automaticamente e foi validado com recursos reais na Cloudflare. Use-o se preferir acompanhar a instalação pelo terminal.

1. Instale **Node.js LTS** (22.13 ou superior; prefira 24 LTS) e **Git** pelos sites oficiais.

2. Abra Terminal no Linux/macOS, Ubuntu no Windows/WSL2, ou PowerShell com Node e Git instalados. Cole uma linha de cada vez e pressione Enter.

```
git clone https://github.com/Quevenapp/Wabotia.git
cd Wabotia
npm ci
npx wrangler login
npm run setup
```

3. No navegador aberto pelo login, escolha a conta Cloudflare correta. Volte ao terminal após a confirmação.

4. Aguarde as cinco etapas: acesso, recursos, banco, secrets e endereço final. Não feche o processo antes da conclusão.

5. Na pasta Wabotia, abra **.secrets.production.local.json** em um editor. Arquivos iniciados por ponto podem estar ocultos. Guarde esse arquivo protegido e copie SETUP_TOKEN somente quando o painel pedir.

6. Abra a URL do Worker, com **/app** ao final. Siga para a página 6.

Teste e domínio próprio

```
npm run setup -- --environment staging
npm run setup -- --domain painel.seudominio.com
```

Use a primeira linha para um ambiente de teste separado; a segunda, se quiser domínio próprio na mesma conta Cloudflare. Não execute o instalador sobre recursos existentes com uma nova MASTER_KEY. Preserve os arquivos .local.json da implantação.

Sua primeira conta

Abra **SUA_URL/app**. A primeira conta é de administrador. As próximas pessoas serão cadastradas dentro do painel.



Tela real de primeira instalação. Os campos foram mantidos vazios.

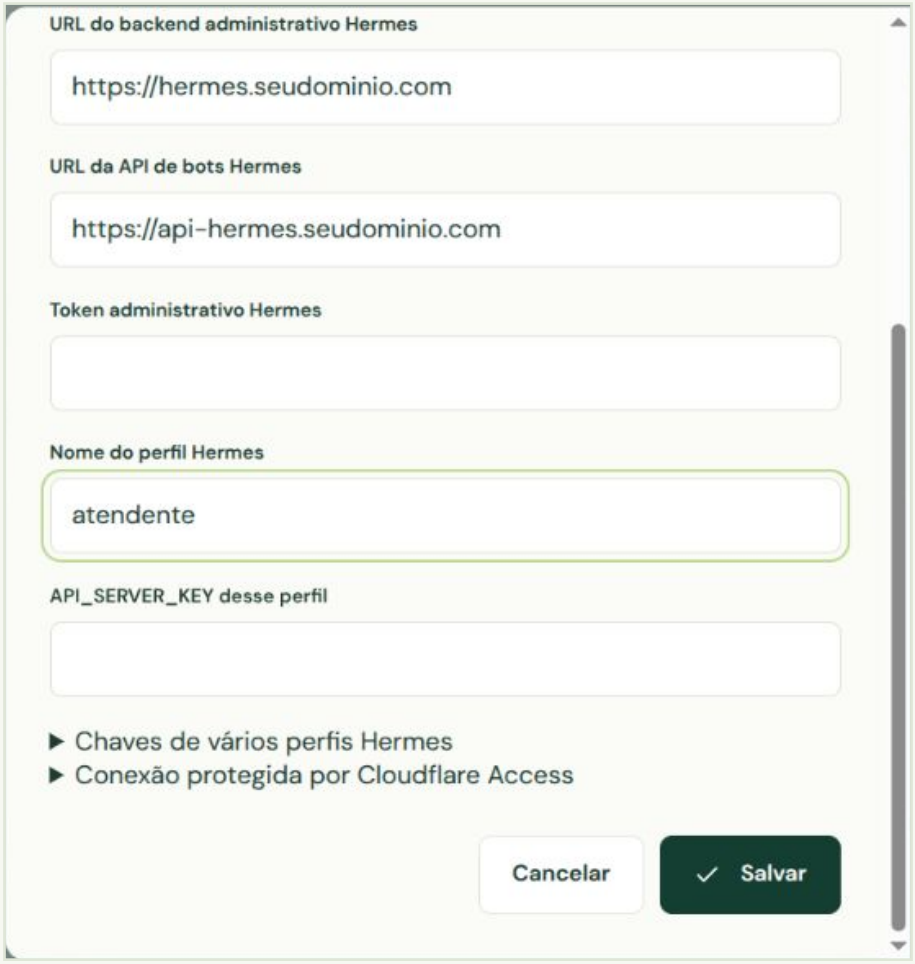
1. Preencha nome, e-mail e senha com pelo menos 12 caracteres. Cole o **SETUP_TOKEN** gerado ou definido no deploy.

2. Clique **Criar conta e entrar**. Depois clique **Criar perfil de atendimento** ou no **+** ao lado do seletor de perfil. Dê um nome, como Atendimento da loja.

3. Mantenha o atendimento pausado. Em **Pessoas**, crie uma conta por atendente e selecione os perfis permitidos.

Conecte o Hermes

O Wabotia descobre os perfis pela API administrativa e executa bots pela API Server nativa. Um Hermes com apenas chat completions pode precisar de atualização compatível.



Conexões > Nova conexão > Hermes. URLs fictícias; tokens vazios.

1. No Hermes, habilite **API_SERVER_ENABLED=true** e defina **API_SERVER_KEY** forte no perfil certo. Reinicie o gateway pelo método que já utiliza.

2. No Wabotia, informe URL HTTPS administrativa, URL da API, token administrativo, nome e chave do perfil. Salve.

3. Clique **Testar conexão** e **Descobrir bots**. Ative a conexão, habilite os bots desejados e selecione o Atendente em **Equipe Bot**.

Referência: API Server na porta 8642; backend administrativo na 9119. Confirme as portas reais. A descoberta verifica idempotência durável. Não use a porta de desenvolvimento do frontend Hermes.

Hermes privado: use túnel

Se o Hermes está em localhost, numa rede privada ou atrás do roteador, o Worker precisa de uma ponte. Use Tunnel protegido por Access. HTTPS autenticado já acessível dispensa o túnel.

```
node scripts/connect-hermes.mjs
```

1. Execute na pasta Wabotia do computador que alcança o Hermes. Informe dois hostnames, por exemplo **hermes.seudominio.com** e **api-hermes.seudominio.com**, as portas e um nome novo para o túnel.

2. O assistente gera **connection.NOME.local.md** e **tunnel.NOME.local.yml**. Abra o primeiro arquivo e siga seus comandos. Ele não altera os bots.

3. Em Cloudflare Zero Trust > **Access > Applications**, crie uma aplicação Self-hosted para cada hostname, com política **Service Auth**.

4. Em **Access > Service Auth > Service Tokens**, crie um token e inclua-o nas duas políticas. Guarde Client ID e Client Secret.

5. Instale cloudflared pelo site oficial. Siga o arquivo gerado para fazer login, criar túnel e rotas DNS. No YAML, complete o UUID e o caminho das credenciais.

6. Inicie o túnel. No formulário Hermes do Wabotia, abra **Conexão protegida por Cloudflare Access** e preencha o Client ID/Secret. Teste.

7. Configure cloudflared como serviço com reinício automático. Preserve volumes no Docker e mantenha o computador ligado.

Backend administrativo: um processo headless compatível pode usar **hermes serve --host 127.0.0.1 --port 9119** e HERMES_DASHBOARD_SESSION_TOKEN no ambiente. Use só se a porta estiver livre. Não crie um segundo gateway nem desative a autenticação de um dashboard público.

Um dashboard público com login/OAuth pode não aceitar Bearer de sessão. A integração precisa de uma API administrativa compatível. No Docker, exponha portas apenas no loopback do host ou use uma rede privada entre os contêineres.

WhatsApp: Meta direto

Escolha este modo para receber a API oficial Meta diretamente no Worker. O Hermes é chamado pelas APIs nativas; não há plugin Wabotia nesse caminho.

1. Na Meta, prepare a aplicação WhatsApp Business e um número de teste. Separe o token, **Phone Number ID**, **WABA ID** e **App Secret**.

2. No Wabotia, abra **Conexões > Nova conexão** e escolha **WhatsApp · API oficial Meta**. Informe o número com DDI, somente dígitos, e os dados da etapa anterior.

3. Crie um **Verify Token** aleatório e preencha o campo. Salve a conexão.

4. Copie o webhook exibido no cartão da conexão. Na aplicação Meta, cadastre essa URL e o mesmo Verify Token. Assine os eventos **messages**.

5. Clique **Testar conexão**, ative o canal e use somente destinatários de teste autorizados na Meta.

6. Teste uma mensagem, a resposta pelo painel e um anexo. Só depois prepare o número de produção.

```
https://SUA_URL/webhooks/meta/ID_DA_CONEXAO
```

Janela, consentimento e templates

Fora da janela de atendimento de 24 horas, esta versão permite notificações de protocolo somente com consentimento e template **UTILITY aprovado**. O corpo deve ter **{{1}}** para protocolo e **{{2}}** para andamento. Configure nome e idioma em Menus e preferências.

> **Um número, um consumidor.** Antes de trocar entre Meta direto, QR ou Hermes, pause o antigo, pare o conector correspondente e confira as pendências em Entregas. Ative o novo modo somente depois da reconciliação.

A confirmação perdida de um envio pode produzir estado unknown. Confira no canal antes de autorizar nova tentativa: ela pode duplicar uma mensagem já recebida.

QR direto ou WhatsApp Hermes

QR direto: processo ao lado do Hermes

1. Crie uma conexão **WhatsApp · QR direto**. Informe o número. Salve e clique **Gerar chave do conector**. Guarde chave e ID da conexão.

2. No host que ficará ligado, abra **connectors/qr**. Copie .env.example para .env e preencha URL HTTPS, ID e chave. Instale e inicie:

```
cd connectors/qr
npm ci
npm start
```

3. Ative a conexão. O QR aparece no cartão do painel. No WhatsApp do celular, abra **Aparelhos conectados** e leia o código.

4. Mantenha o processo ligado. Preserve **data/**, com sessão e diário; configure reinício e backup. Não precisa abrir portas de entrada nem túnel para esse conector.

Em Docker, após preencher .env nessa pasta, use **docker compose up -d --build**. Preserve o volume qr-data ao atualizar. Baileys é um cliente não oficial; o pareamento deve ser validado na sua conta.

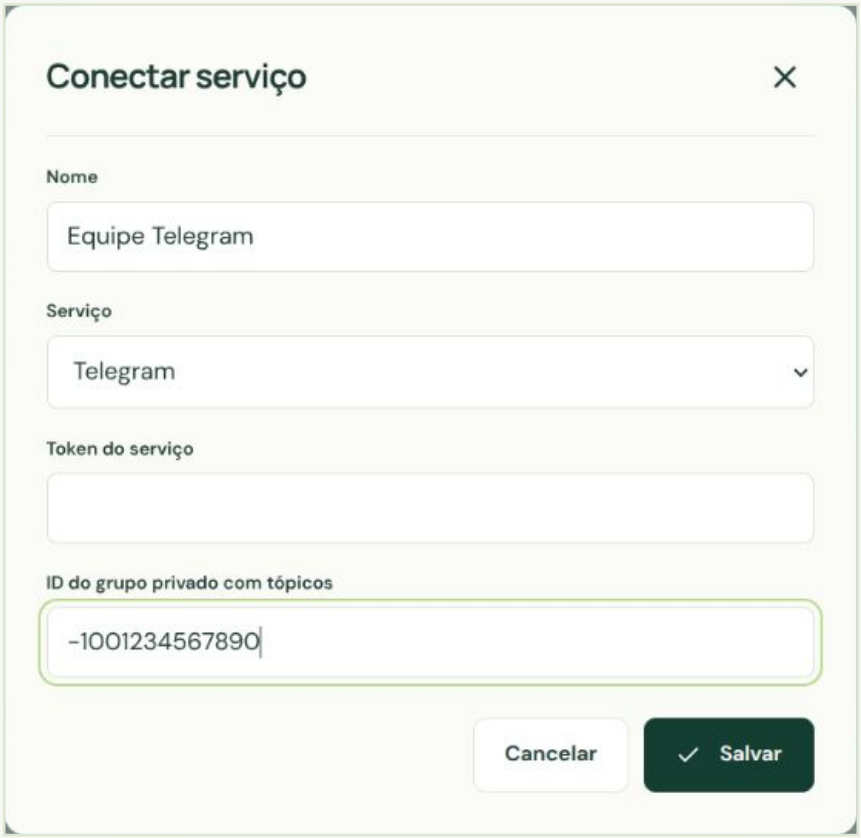
WhatsApp que já está ligado no Hermes

Mantenha Meta/QR configurados no Hermes e escolha **WhatsApp · pelo Hermes** no painel. Esse caminho precisa do transporte mínimo e da extensão compatível de entrada exclusiva no gateway.

Com o gateway parado e o perfil copiado para backup, siga o README de **connectors/hermes**: conferir a revisão, testar o patch com git apply --check, instalar no perfil existente e habilitar wabotia_transport em in_process. Reinicie pelo processo que já utiliza.

Não misture os caminhos. WhatsApp direto e Telegram direto dispensam esse plugin. A extensão do Hermes requer manutenção técnica e revalidação quando o gateway é atualizado; não aplique em outra revisão sem conferir compatibilidade.

Telegram para sua equipe



Nova conexão > Telegram. O grupo é fictício e o token não está preenchido.

1. Crie um bot exclusivo no **@BotFather** com /newbot. Não reutilize o bot do gateway Hermes.

2. Crie um **grupo privado com tópicos**. Adicione o bot como administrador, com permissão de gerenciar tópicos e enviar mensagens e anexos.

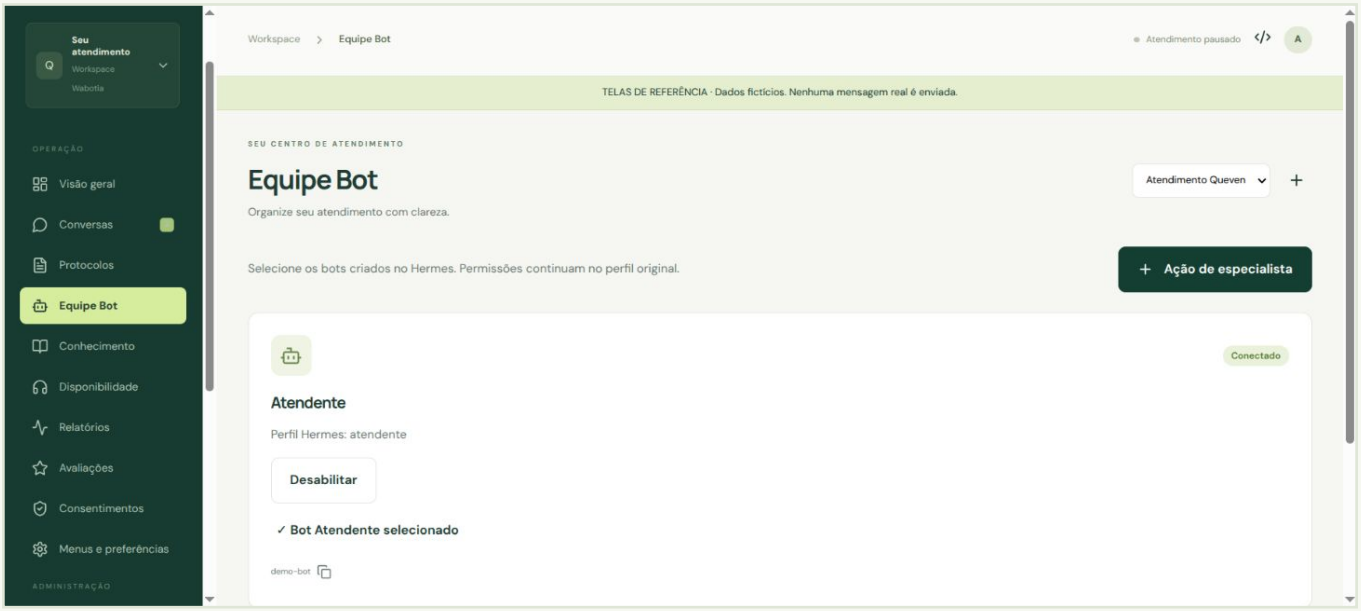
3. Obtenha o ID numérico do grupo, geralmente -100...; o Bot API getUpdates pode mostrar esse ID antes de configurar o webhook.

4. No painel, informe token e grupo, salve, teste e clique **Ligar recebimento Telegram**. Ative a conexão. Em Menus e preferências, cole os IDs da conexão e do grupo.

5. Cada pessoa envia **/identificar** no grupo. Vincule o ID retornado à conta correta em Pessoas. Depois use **/presente**.

O Telegram conversa diretamente com o Worker. Não instala plugin no Hermes. Vincule apenas o ID verificado da pessoa; não use bots terceiros para acessar os dados do grupo.

Prepare os bots e os menus



Equipe Bot com um perfil de demonstração. Os dados são fictícios.

1. Em **Equipe Bot**, habilite os perfis descobertos e selecione quem será o **Atendente**. Só habilite perfis cuja API_SERVER_KEY foi cadastrada.

2. Clique **Ação de especialista**. Escolha o bot, descreva quando consultá-lo e a ação permitida. Os modelos e ferramentas continuam configurados no Hermes.

3. Abra **Conhecimento** e cadastre informações úteis: texto, URL pública, PDF com texto ou DOCX. Espere a importação ficar disponível. Arquivos têm limite de 20 MB.

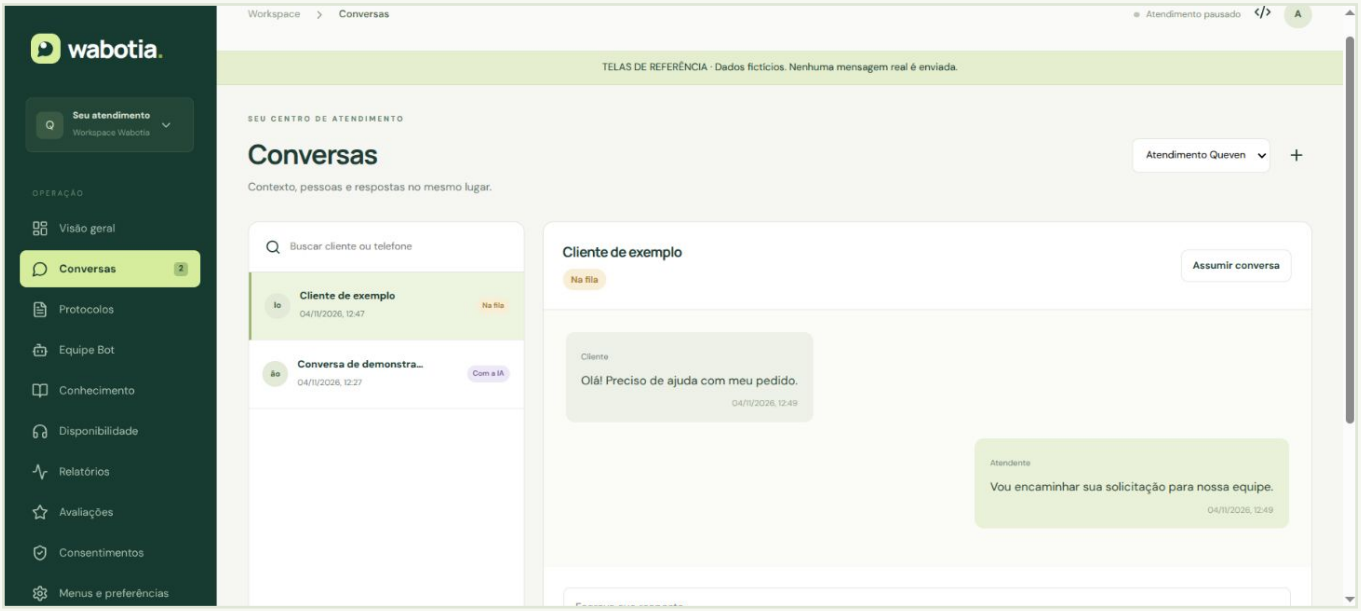
4. Em **Menus e preferências**, configure prazo, instruções, avaliações e opções de menu. Se necessário, use o editor JSON para aplicativos e submenus. JEV é opcional.

5. Cadastre pessoas e permissões por perfil. Em **Disponibilidade**, registre presença antes de abrir o atendimento.

Arquivos de bots: precisam ser exportados como artefatos da API nativa. Caminhos locais não viram links públicos. Downloads de uso único interrompidos exigem nova exportação; o Wabotia não os repete automaticamente.

PDF digitalizado sem texto não tem OCR nesta versão. Vídeos precisam de legendas públicas/autorizadas; também é possível importar TXT, VTT ou SRT.

Faça a primeira conversa



Tela de atendimento com mensagens fictícias. Assumir invalida a resposta automática anterior.

1. Use um número de teste autorizado. Envie uma pergunta que possa ser respondida com o conhecimento cadastrado.

2. Abra a conversa e clique **Assumir**. Confira que o responsável é sua conta e responda pelo painel.

3. Teste nota interna, anexo, transferência e abertura de protocolo. Nota interna não deve ir ao cliente.

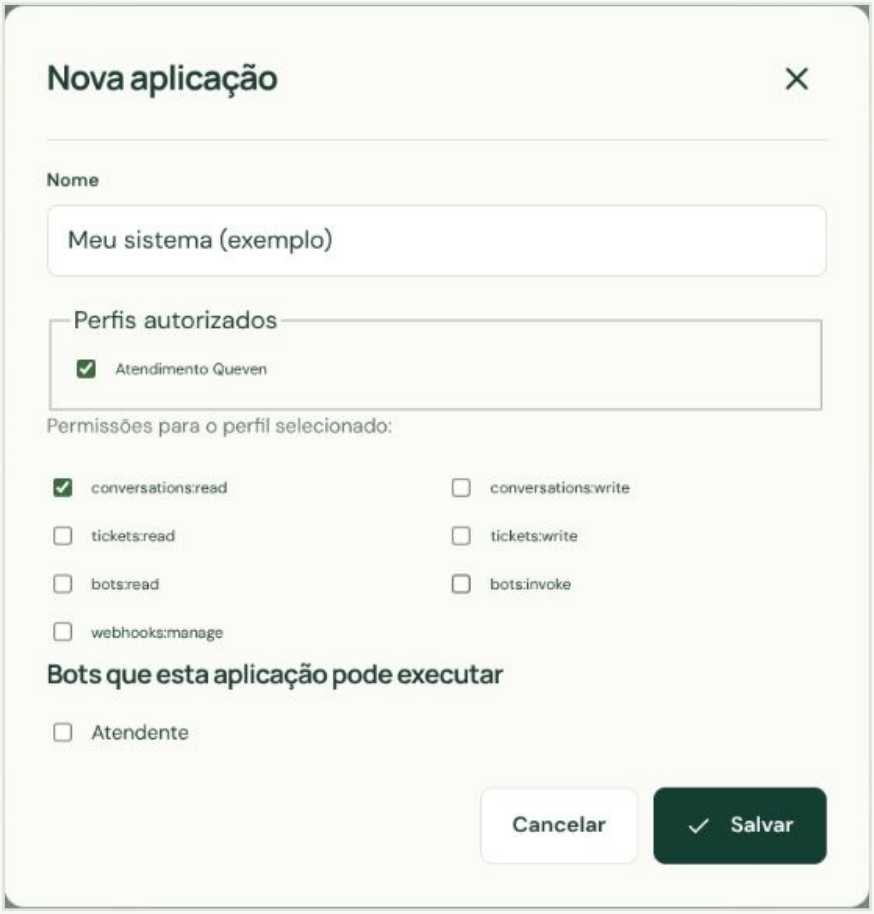
4. No Telegram, vá ao tópico da conversa e use /assumir antes de /responder. Confira o resultado também no painel.

Comando Telegram	Uso
/presente ou /ausente	Registra disponibilidade.
/assumir e /responder texto	Assume a conversa e responde ao cliente.
/nota texto e /transferir ID	Registra nota interna e transfere para atendente autorizado.
/encerrar e /retomar	Conclui ou devolve o atendimento à automação.
/identificar e /ajuda	Mostra sua identidade ou a lista de comandos.

No modo por comando, texto comum vira nota interna. No modo direto, o responsável responde sem /responder conforme as regras do tópico. Confira esse comportamento com mensagens fictícias antes de usar.

Integre outros sistemas

A API é para comunicação entre servidores. Seu programa usa uma chave própria e só opera os perfis, bots e ações autorizados.



API e integrações > Nova aplicação. Exemplo de seleção de permissões.

1. Cadastre uma aplicação, escolha os perfis e marque as permissões necessárias. Selecione os bots que ela poderá executar.

2. Clique **Gerar chave**, escolha validade e copie o valor exibido para o backend do integrador. Para rotação, emita outra e depois revogue a antiga.

3. Abra **/api-docs** na sua instalação. Há OpenAPI 3.1 e exemplos em cURL, JavaScript e Python no repositório.

4. Cadastre um webhook HTTPS para eventos. Guarde o segredo HMAC e verifique assinatura, timestamp e ID do evento. O destinatário precisa deduplicar.

Escritas exigem Idempotency-Key. Bots retornam 202 e um ID para acompanhamento. Repetir a chave com outro corpo gera conflito; timeout não deve disparar uma nova execução automaticamente.

Backup e manutenção

Copiar seus dados

1. Pause o atendimento e as entradas dos canais. Pare QR/transporte para não gravar durante a cópia. Preserve os diários.

2. Na pasta Wabotia, execute o comando. Use o arquivo de configuração da implantação correta.

```
node scripts/backup.mjs wrangler.production.local.json backup-seguro
```

O backup inclui banco e mídias. Só considere concluído quando manifest.json tiver **complete: true**. Guarde a pasta protegida e a **MASTER_KEY original separadamente**. Depois retome os canais.

Restaurar em recursos novos

Crie um D1 vazio e um R2 privado novo. Faça uma configuração local apontando para esses recursos, sem sobrescrever a produção. Então execute:

```
node scripts/restore.mjs wrangler.restore.local.json backup-seguro --empty-target
```

O script recusa banco preenchido. Configure a MASTER_KEY original, publique a mesma versão e teste dados e arquivos antes de mover o domínio. Pare a instalação antiga antes de ligar os mesmos canais na restaurada.

Diagnóstico

```
node scripts/diagnose.mjs https://SUA_URL
```

Sintoma	O que conferir
Hermes offline	Processo, portas, túnel, Access, URLs e chaves de cada perfil.
401 / 403 / 404	Validade da chave, permissões, perfil e dono do recurso.
429	Respeite Retry-After e reduza a frequência.
unknown	Confira Hermes/canal e use Execuções ou Entregas para reconciliar.

Antes de atender clientes

O núcleo e a publicação já foram testados. Seus perfis e canais precisam passar por homologação com credenciais e destinatários autorizados. Não confunda uma tela de demonstração com um canal conectado.

Execute na sua instalação de teste

1. Envie e receba texto e anexo em cada canal escolhido. Confira protocolo, prazo, consentimento, avaliações, presença, notas e transferência.

2. Peça uma resposta lenta ao bot e assuma no meio da execução. A resposta automática antiga não deve chegar ao cliente.

3. Desligue e religue Hermes, túnel e conector. O painel deve continuar acessível; confira a retomada e os estados sem confirmação.

4. Repita uma mensagem/webhook de teste. Confira deduplicação, assinatura e recuperação. Teste também uma chave expirada/revogada.

5. Valide backup e restauração antes de depender da instalação. Só então ative o atendimento para clientes.

Evidência desta edição: 27 testes em workerd, 3 testes Python do transporte, 20 verificações HTTP em staging, exemplos em três linguagens e restauração remota D1/FTS/R2. Os testes externos usaram simulações ou dados fictícios. **Meta, QR, Telegram e Hermes reais ainda não foram homologados nesta implantação.**

Onde encontrar os detalhes

Site e guia atualizado

Código aberto e instruções completas

Matriz de validação e limites

Operação, atualização e recuperação

Uma organização por implantação. Fuso America/Sao_Paulo. O histórico antigo não é importado automaticamente. Telas capturadas em 04/11/2026; menus de fornecedores podem mudar. Wabotia é independente de Cloudflare, Meta, Telegram e NousResearch.